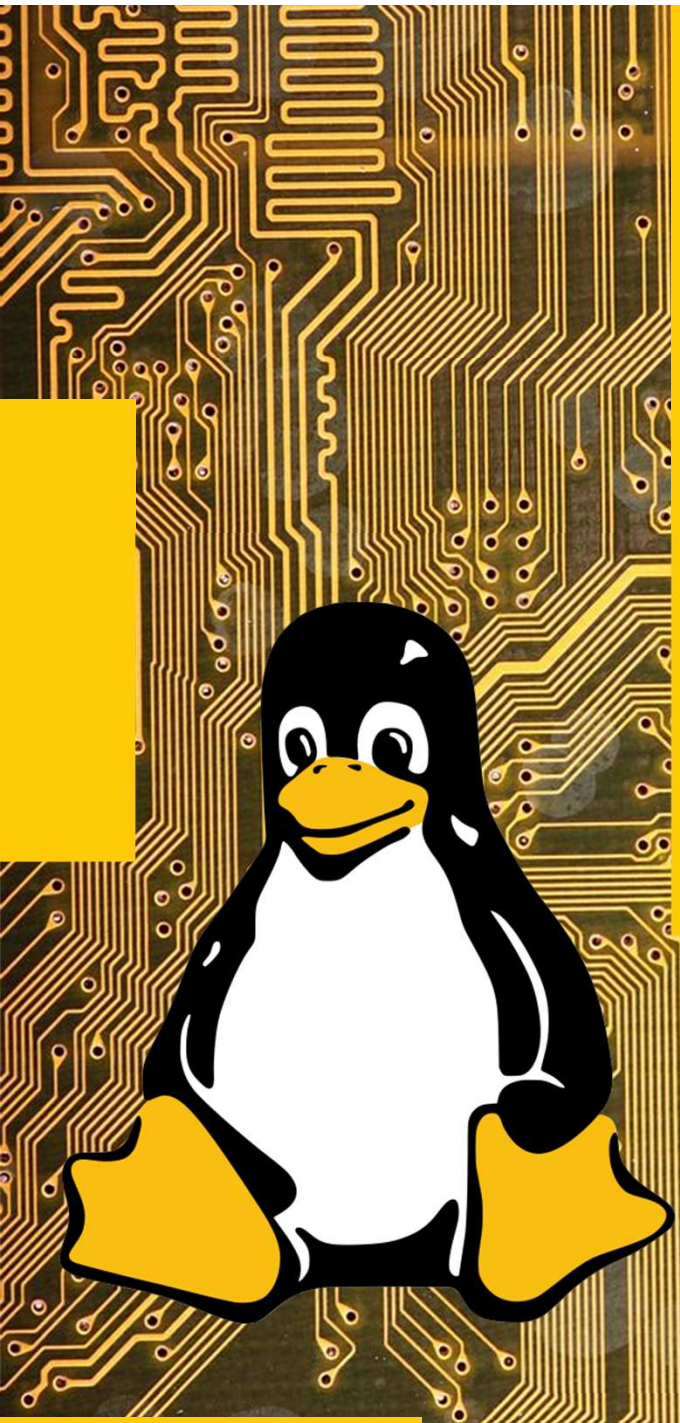


INHA UNIVERSITY IN TASHKENT

SOC 3010 OPERATING SYSTEMS

Course Organization

INSTRUCTOR: Jalolliddin Yusupov
PROFESSOR OF SOCIE (Room A406)

CREDITS: 3 (You'll earn these)
CLASS HOURS: 3 (Where the magic happens)
HOMEWORK HOURS: ∞ (Where the debugging happens)

(Serious inquiries only. For panic attacks, please press Ctrl+Alt+Del)

Yusupov Jalolliddin

SOC 3010 - Operating Systems
SOC 2040 - System Programming
SOC 4190 - Blockchain Basics & Platforms





Contact Details:

- **EMAIL:** j.yusupov@inha.uz
- **Room No.:** A406/4
- **Interaction:** through eClass

Office Hours:

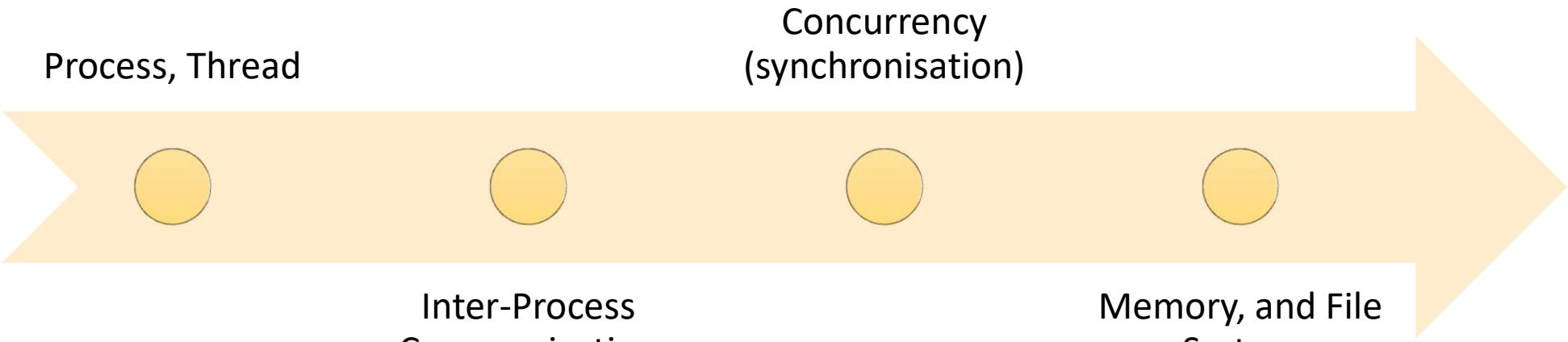
- **Tuesday** 13.00-14.00
- **Thursday** (by appointment)

Interests: Hiking, Espresso enthusiast

Course Syllabus: COURSE DESCRIPTION

This course provides the knowledge required to understand how a **modern** operating system works from the point of view of the programs that run on it

The course begins by completing the process and signal **foundations** from **System Programming** and works throughout on the Linux system: every concept taught in the lecture is observed, measured and implemented by the student on their own machine.



Course Syllabus: COURSE LEARNING OBJECTIVES & OUTCOMES

Understand basic concepts of operating system

- (Relation with Program Outcomes PO 2)

Implementing a simple operating system

- (Relation with Program Outcomes PO 3)

At the end of the course, students should have

PO2:	ABEEK PO2 : Analysis of experimental data- An ability to understand and analyze data, and plan and execute experiments
PO3:	ABEEK PO3 : Problem formulation & solution - An ability to define, formulate and solve engineering problems

TO BE FOLLOWED VERY STRICTLY

1. Professional Conduct & Etiquette

- Zero Distractions (No Phones during Lectures)
- Punctuality (Attendance will not be registered if you arrive more than 10 minutes after class starts)

2. Technology Policy (Laptop Use)

- Course-Related Activity Only

3. Attendance & Exam Eligibility

4. Academic Integrity (Code & Assignments)

- Original Work



Course Syllabus: COURSE LEARNING OBJECTIVES & OUTCOMES

PREREQUISITES

- [MANDATORY] c programming, data structures

RECOMMENDED COURSES AFTER THIS COURSE

- Computer security, embedded systems

COURSE SOFTWARE OR TOOL

- linux (ubuntu linux distribution)

TEXTBOOK

- **REQUIRED** — Operating Systems: Three Easy Pieces (OSTEP) Author: Remzi H. et.al. Publication Year: 2023 (v1.10)
  — freely available at www.ostep.org
- **RECOMMENDED** — Operating System Concepts, 10th Edition Author: Silberschatz. ISBN: 978-1118063330 (each week lists both OSTEP and Silberschatz chapters; you may read either)
- **RECOMMENDED –** (TPLI) The Linux Programming Interface: A Linux and UNIX System Programming Handbook. ISBN: 1593272200

Grading Policy

Methods of Evaluation:
Assessment will be made on the basis of one midterm Exam, One Final Examination, Weekly Quizzes, Home Assignments and Term Project

Evaluation Criteria:

	100%
Attendance	10%
Weekly Quizzes (10)	10%
Home Assignment (4)	10%
Term Project Work (1)	20%
Mid-Term Exam	25%
Final Exam	25%

(Relative) GRADES AWARDED: A+, A0, B+, B0, C+, C0, D+, D0, F (FAIL)

TEAM FORMATION INSTRUCTIONS FOR TERM PROJECT

- you are required to form teams comprising a team leader and members for carrying out **Term Project (details will be provided).**
- E-Class
- Late submissions will not be accepted
- adhere to the deadline strictly
- You may use AI tools for studying and as a coding assistant. Any use must be disclosed in the submission. Grades are decided by what the student can explain and modify in person: sampled oral checks on home assignments and the individual oral defence at the project demonstration. Submitted work that the student cannot explain is graded as not done.

WEEK I Course Topics & Weekly Lecture Breakdown

Lecture Topic: Processes and Signals, Completed (OSTEP Ch 4, 5, 6)

- Review and deepening of System Programming: process states, fork and process graphs, waitpid and the status macros, zombies and orphans, signals and handlers.
- **Continue with what was missing**: the exec family and why fork and exec are separate; sigaction, signal masks, interrupted system calls, writing a safe signal handler, reaping children with SIGCHLD; the duties of process 1 and shutting a program down cleanly.

Lab Objective

- Predict on paper the output of a generated fork/exec/signal program, then run it and explain every difference.
- Write a minimal shell that runs a command and reports its exit status.

WEEK II Course Topics & Weekly Lecture Breakdown

Lecture Topic: The Shell and CPU Scheduling
(TLPI Ch 34; OSTEP Ch 7, 8)

- The shell: exit codes, quoting, subshells, sessions and process groups, job control, daemons, traps, and **bash script programming**. Next — scheduling: how the OS regains the CPU; turnaround and response time; first-come-first-served, shortest job first, round robin, priorities; the multi-level feedback queue; the Linux scheduler and control-group CPU limits.

Lab Objective

- Extend the shell with signal handling and job control (no zombie processes); write a shell script that supervises a service.
- Compute schedules by hand for a given set of jobs and verify with the course simulator.

WEEK III Course Topics & Weekly Lecture Breakdown

Lecture Topic: Inter-Process Communication - Pipes and FIFOs (TLPI Ch 44)

- Why processes cannot see each other's memory; message passing and shared memory. Pipes: creation, inheritance across fork, closing unused ends, blocking and end-of-file rules, atomic writes, the SIGPIPE signal; redirection with dup2. Named pipes (FIFOs) and a simple client-server built on them; the multi-stage pipeline.

Lab Objective

- Add multi-stage pipelines and redirection to the shell, with no leaked file descriptors.
- Find and repair the fault in a supplied program that hangs on a pipe.

WEEK IV Course Topics & Weekly Lecture Breakdown

Lecture Topic: IPC - Message Queues, Shared Memory and Semaphores (TLPI Ch 45-54)

- Message queues: POSIX and System V interfaces, message priorities, the persistence of System V objects. Shared memory: creating and mapping a shared region. The lost-update problem demonstrated, and the first semaphore as the lock that protects shared memory between processes. Cleaning up IPC objects.

Lab Objective

- Make a shared-memory counter correct with a semaphore and prove it under load; a message-queue logger with priorities.

WEEK V Course Topics & Weekly Lecture Breakdown

Lecture Topic: Sockets and Concurrent Servers (TLPI Ch 56-61)

- The socket system calls, the listen queue, and message framing over a byte stream. Handling the death of a peer and the reuse of an address. Unix domain sockets, peer credentials, and passing an open file between processes. Server architectures: one client at a time, a process per client, a pool of processes; select and poll.

Lab objective

- Write a multi-client chat server using a process per client, sharing the room state through shared memory and a semaphore.

WEEK VI Course Topics & Weekly Lecture Breakdown

Lecture Topic: Synchronisation with Semaphores - (OSTEP Ch 31; Downey Ch 1-4)

- Part 1 — the semaphore patterns with processes and shared memory: mutual exclusion, multiplex, signalling, barrier; ordering problems and precedence graphs; Part 2 — workshop on paper: invariants and trace tables, filling in the missing waits and signals, enumerating the possible interleavings, the catalogue of classic semaphore bugs.

Lab Objective

- Implement the bounded buffer in shared memory with semaphores (no message lost or duplicated, no busy waiting); a precedence-graph program from a generated graph; trace a generated semaphore program by hand, then run it.

WEEK VII Course Topics & Weekly Lecture Breakdown

Lecture Topic: Synchronisation with Semaphores and the Classical Problems (OSTEP Ch 31)

- Part 1 — the lightswitch pattern; the readers and writers problem, writer starvation measured and fixed with a turnstile; categorical mutual exclusion; the sleeping barber with one and with several barbers. Part 2 — the dining philosophers, why the naive solution deadlocks, the starvation scenario, resource allocation graphs; midterm workshop on process graphs, pipes and signals..

Lab Objective

- Readers-writers under two policies with a starvation measurement and the turnstile fix; the sleeping barber with several barbers; the dining philosophers made to deadlock and then fixed.

WEEK VIII Course Topics & Weekly Lecture Breakdown

It's midterm season. Enough said.



WEEK IX Course Topics & Weekly Lecture Breakdown

Lecture Topic: Threads and Locks (OSTEP Ch 26, 27, 28)

- What threads share and what they keep private; the POSIX thread interface; threads as implemented on Linux. Uncontrolled scheduling and the race condition at the instruction level; critical sections and atomicity.

Lab Objective

- Find, explain and repair the data races planted in a supplied program, keeping the corrected program fast.

WEEK X Course Topics & Weekly Lecture Breakdown

Lecture Topic: Condition Variables, Read-Write Locks and the Two Idioms (OSTEP Ch 30, 31)

- Waiting for a condition: condition variables, why the wait must be inside a loop, signalling one waiter or all. The bounded buffer derived again with threads and compared with the semaphore solution of Week 6.

Lab Objective

- Replace busy waiting in a thread pool with condition variables; implement readers-writers with read-write locks and compare with the semaphore version.

WEEK XI Course Topics & Weekly Lecture Breakdown

Lecture Topic: Concurrency Bugs, Deadlock and Event-Based Concurrency (OSTEP Ch 32, 33)

- Concurrency bugs that are not deadlocks. The four conditions for deadlock and the resource allocation graph. Livelock, starvation, priority inversion. Event-based concurrency: the event loop and its state machines; when threads are the wrong tool.

Lab Objective

- Diagnose a deadlocked program from its core dump and repair it; rebuild the chat server on an event loop and compare it with the process-per-client version.

WEEK XII Course Topics & Weekly Lecture Breakdown

Lecture Topic: Memory - Address Spaces, Paging and the TLB (OSTEP Ch 13-15, 18-20)

- The address space and the illusion it provides. The memory interface and the common memory errors. Address translation. Paging: pages and frames, the page table and its entries. The address space of a Linux process, memory mapping, and the page fault and its kinds.

Lab Objective

- Translate addresses by hand for a given page table, then verify with the course simulator.
- Write a program that provokes each kind of page fault on demand and report the measurements.

WEEK XIII Course Topics & Weekly Lecture Breakdown

Lecture Topic: Memory - Swapping, Allocation and the Out-of-Memory Killer (OSTEP Ch 17, 21, 22)

- Memory beyond the physical: swap space and the page fault that reaches the disk. Free space management: splitting, coalescing and the fitting strategies; the buddy allocator; what the C library allocator really does. Memory overcommit and how the kernel chooses a process to kill.

Lab Objective

- Simulate the replacement policies by hand on a given reference string.
- Implement a small memory allocator and test it.
- Cause an out-of-memory kill inside a memory-limited group and explain the kernel's choice.

WEEK XIV Course Topics & Weekly Lecture Breakdown

Lecture Topic: Files, Directories and File System Implementation (OSTEP Ch 39, 40)

- Files, directories and the file interface. The inode and why a file name is not the file. Hard links, symbolic links, and what deleting a file really does. The layout of a simple file system: the superblock, the bitmaps, the inode table and the data region. Indexing a file with indirect blocks and with extents.

Lab Objective

- Answer a set of forensic questions about a supplied file system image (inode numbers, link counts, file sizes).
- Recover a deleted file that is still held open by a running process.

WEEK XV Course Topics & Weekly Lecture Breakdown

FINAL EXAM

WEEK XVI Course Topics & Weekly Lecture Breakdown

MAKUP ASSESMENT

Attendance Policy

1. All students should attend all classes
2. Students who are absent without valid reason for more than 25% of the assigned hours for the course will be given a Grade “F

Academic Dishonesty Policy

- In order for instructors to **fairly** assess the quality and quantity of a student's learning (through course grades) as determined by work that students represent **as their own,** a relationship of trust between instructor and student is essential.
- Because violations of academic integrity most often involve, but are not limited to, **efforts to deceive instructors**, they represent a breach of the **trust relationship between instructor** and student, and undermine the core values of the university.
- For these reasons, the University and its instructors treat issues of academic dishonesty **as serious violations** of academic trust, and conduct rigorous investigations of students suspected of committing such acts.

ACTS OF ACADEMIC DISHONESTY INCLUDE, BUT ARE NOT LIMITED TO, THE FOLLOWING:

- the **illegitimate** use of materials in any form during a quiz or examination
- **copying** answers from the quiz or examination paper of another student
- **plagiarizing** (submitting as one's own ideas the work of another) or falsifying materials or information used in the completion of any assignment which is graded or evaluated as the student's individual effort
- submitting the **same work** for more than one course without the consent of the instructors of each course in which the work is submitted
- copying material from a web page and submitting it as **one's own work**
- **quoting extensively** from a document without making proper references to the source

- If a student is found committing such acts in a quiz or home assignment or exam or term paper or term project he/she will be given a grade 0 in that part of the course



Questions?